Лабораторна робота №9

Тема: React State та Effects: Context API, useReducer, useEffect і Custom Hooks.

Мета: навчитися організовувати стан React-застосунку, коли одного локального `useState` стає недостатньо. Опанувати підняття стану, Context API, `useReducer`, побічні ефекти, роботу з `localStorage` та створення власних hooks.

Лабораторна є продовженням лабораторної роботи №8. Необхідно продовжити розробку застосунку **Study Planner** без втрати попереднього функціоналу.

Рекомендована feature-гілка: `feature/state-effects`.

Вимоги до звіту

1. Використовувати React 19.x та Node.js 22.22 або новішу версію.
2. Використовувати TypeScript.
3. Використовувати функціональні компоненти та hooks.
4. Використати Context API для спільного стану.
5. Використати `useReducer` щонайменше для одного набору пов'язаних змін стану.
6. Використати `useEffect` для роботи з побічним ефектом.
7. Реалізувати збереження даних у `localStorage`.
8. Винести повторно використовувану логіку в Custom Hook.
9. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS.
10. Якщо обрано SCSS, дотримуватися BEM-патерну для класів.
11. Дотримуватися ESLint та єдиного стилю коду.
12. Код повинен бути розміщений в особистому GitHub репозиторії студента.
13. Дотримуватися Conventional Commits.

Теоретична частина

Опрацювати:

1. Managing State.
2. Sharing State Between Components.
3. Extracting State Logic into a Reducer.
4. Scaling Up with Reducer and Context.
5. Synchronizing with Effects.
6. Reusing Logic with Custom Hooks.

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;
2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**:

- стилі кожного компонента або логічного UI-блоку зберігати у власному `.scss` файлі поруч із компонентом;
- не зберігати стилі компонентів у загальному `App.css` або іншому великому спільному файлі;
- класи повинні відповідати BEM;
- глобальні стилі залишати лише для справді глобальних речей: `body`, базові стилі, шрифти, reset тощо.

Практична частина

1. Розвиток Study Planner

Продовжити проєкт з лабораторної роботи №8. Не видаляти вже реалізований функціонал.

Функціонал №8 вже вважається виконаним і не є окремим завданням цієї роботи: * створення завдання; * зміна стану виконання; * видалення; * базова фільтрація; * статистика; * повідомлення для порожнього списку; * індикація активного фільтра; * передача state та callback через props там, де вона вже реалізована.

У №9 потрібно не повторювати ці рішення, а **перебудувати організацію state та додати persistence**.

2. Refactoring state через useReducer

Замінити основний `tasks` state з лабораторної №8 на `useReducer`.

Reducer повинен підтримувати щонайменше такі дії:

- `add` — додавання завдання;
- `toggle` — зміна completed;
- `update` — редагування завдання;
- `requestDelete` — запуск Undoable Delete;

- `confirmDelete` — остаточне видалення після завершення таймера;
- `undoDelete` — скасування запланованого видалення;
- `clearCompleted` — масове видалення виконаних, лише якщо для нього реалізовано власний undo-flow;
- `reset` — повернення до початкових даних.

Для кожної дії використати TypeScript-тип action з необхідними payload.

Reducer повинен залишатися чистою функцією: * не виконувати HTTP-запити; * не звертатися до DOM; * не працювати безпосередньо з `localStorage`; * не містити UI-логіку.

3. Редагування завдання

Додати можливість редагувати існуюче завдання.

Студент повинен мати можливість змінити щонайменше:

- `title`;
- `description`;
- `priority`;
- `category`.

Можна повторно використати `TaskForm` з лабораторної №8, додавши режим редагування.

При збереженні зміни повинні проходити через `dispatch` та reducer, а не змінювати об'єкт завдання напряму.

4. Context API

Створити `TaskContext` та `TaskProvider`.

Context повинен надати компонентам доступ до спільного стану та операцій без передачі `tasks`, `filter` і callback-функцій через декілька рівнів props.

Рекомендований API Context:

```
{
  tasks,
  filter,
  setFilter,
  addTask,
  updateTask,
  toggleTask,
  requestDelete,
  undoDelete,
  clearCompleted,
  resetTasks,
```

```
  toast
}
```

Конкретна форма API може відрізнятися. Головне — компоненти не повинні знати деталі reducer та timer-логіки.

Реалізація може відрізнятися, але компоненти не повинні отримувати один і той самий state через зайві рівні props.

5. Custom Hooks

Створити щонайменше два власні hooks:

- `useTaskContext()` — зручний доступ до `TaskContext`;
- `useLocalStorage<T>()` — повторно використовувана робота зі сховищем браузера, наприклад для активного фільтра.

`useTaskContext()` повинен мати зрозумілу помилку, якщо його використали поза `TaskProvider`.

`tasks` повинні залишатися єдиним state, яким керує reducer. `useLocalStorage` не повинен створювати друге джерело правди для списку завдань.

Не потрібно створювати окремий hook для кожної дрібної функції.

6. localStorage та useEffect

Зберігати `tasks` у `localStorage`.

Рекомендований сценарій:

1. під час ініціалізації reducer прочитати збережені задачі, якщо вони є;
2. якщо даних немає або вони пошкоджені, використати початковий список;
3. після зміни `tasks` синхронізувати актуальне значення з `localStorage` через `useEffect`;
4. обробити помилку `JSON.parse`, щоб пошкоджені дані не ламали застосунок;
5. активний фільтр можна зберігати через `useLocalStorage`.

У результаті reducer залишається єдиним джерелом правди для `tasks`, а `localStorage` лише зберігає їх між перезавантаженнями.

Не використовувати `useEffect` для обчислення filtered list або статистики — це derived state і він повинен обчислюватися під час render.

7. Синхронізація з браузером

Додати невеликий Effect, який синхронізує заголовок вкладки браузера з кількістю невиконаних завдань.

Наприклад:

```
Study Planner — 3 active tasks
```

Для цього використати `document.title` у `useEffect`.

Це окремий приклад синхронізації React state із зовнішнім API браузера.

8. Toast-повідомлення

Створити власний компонент `Toast` без сторонньої бібліотеки.

Toast використовувати для короткого зворотного зв’язку після основних дій:

- успішне створення завдання;
- успішне редагування;
- остаточне видалення;
- помилка операції або неможливість виконати дію.

Toast повинен:

- мати зрозумілий текст;
- автоматично зникати через короткий проміжок часу;
- не блокувати роботу з рештою інтерфейсу.

Для стилізації не потрібно створювати окрему notification-бібліотеку.

9. Undoable Delete

Замість миттєвого видалення реалізувати **Undoable Delete**.

Після натискання «Видалити»:

1. не видаляти завдання остаточно одразу;
2. позначити його як таке, що очікує видалення;
3. показати Toast із повідомленням на кшталт «Завдання буде видалено»;
4. запустити таймер на **5 секунд**;
5. показувати в Toast візуальний прогрес таймера;
6. надати кнопку **«Скасувати»** / **«Undo»**;
7. якщо користувач натиснув Undo — скасувати таймер і повернути завдання до звичайного стану;
8. якщо таймер завершився — виконати `confirmDelete` і видалити завдання остаточно.

Під час очікування видалення завдання повинно мати помітний візуальний стан,

наприклад приглушення або напис «Буде видалено».

Оскільки таймер є side effect, його не можна створювати всередині reducer. Reducer лише змінює state, а `useEffect` відповідає за запуск та очищення таймера.

Під час unmount компонента або зміни pending deletion попередній timer повинен бути очищений через `clearTimeout`.

На базовому рівні достатньо підтримувати **одне активне Undoable Delete одночасно**. Підтримка черги з кількох одночасних видалень — додаткове завдання.

Видалення повинно залишатися undoable у межах поточної сесії. Після перезавантаження сторінки незавершене видалення не потрібно відновлювати.

Візуальний приклад Undoable Delete

Щоб було зрозуміло, як очікується поведінка механізму Undoable Delete, нижче наведено короткий візуальний приклад.

[Undoable Delete — візуальний приклад] | *assets/undoable-delete-example.gif*

У прикладі показано два сценарії:

- натискання «Delete» → запускається 5-секундний таймер → користувач натискає «Undo» → таймер скасовується, завдання повертається у список;
- натискання «Delete» → таймер завершується → виконується остаточне видалення.

Це лише **приклад поведінки та UX**, а не готова реалізація, яку потрібно копіювати один в один.

10. UX після refactoring

Після перенесення state у reducer і Context зберегти функціонал №8:

- фільтрацію;
- derived statistics;
- повідомлення для порожнього списку;
- візуальний стан виконаного завдання.

Ці можливості потрібно зберегти, але не реалізовувати вдруге окремою логікою.

Додати до UI:

- кнопку «Очистити виконані» — лише як додаткову можливість із власним undo-flow;
- кнопку «Скинути дані»;
- режим редагування завдання;
- Toast після create/edit/delete;

- Undoable Delete з прогресом 5 секунд.

11. Додаткові завдання

Для підвищення оцінки можна реалізувати один або декілька пунктів:

1. синхронізацію змін між відкритими вкладками через `storage` event;
2. відновлення останньої видаленої задачі;
3. окремий `ThemeContext` зі збереженням теми;
4. підтримку кількох одночасних Undoable Delete;
5. масове видалення виконаних із можливістю Undo;
6. оптимізацію `Context value`, щоб не створювати новий object без потреби;
7. використання `useEffectEvent` там, де це справді доречно, з поясненням, чому звичайний Effect має інші залежності. == Рекомендована структура

```
src/
├─── components/
│    ├─── TaskForm.tsx
│    └─── Toast.tsx
├─── context/
│    └─── TaskContext.tsx
├─── hooks/
│    ├─── useTaskContext.ts
│    └─── useLocalStorage.ts
├─── reducers/
│    └─── taskReducer.ts
├─── types/
├─── data/
├─── App.tsx
└─── main.tsx
```

Структура може бути змінена, якщо відповідальність модулів залишається зрозумілою. Не потрібно створювати окремий модуль для кожної функції або будувати складну feature-first/FSD-архітектуру: у цій лабораторній головною метою є організація state та side effects.

Демонстрація на захисті

За час захисту необхідно продемонструвати:

1. створення або редагування завдання та Toast;
2. запуск Undoable Delete і кнопку Undo;
3. остаточне видалення після завершення таймера;
4. перезавантаження сторінки та відновлення даних з `localStorage`;

5. роботу компонента, який отримує state або actions через Context;
6. у коді показати `useReducer`, `useEffect`, `useTaskContext` та `useLocalStorage`.

Не потрібно демонструвати всі додаткові можливості.

Приклад Conventional Commit:

```
feat(lab-09): add reducer context and persistence
```

Контрольні запитання

1. Коли state доцільно піднімати у батьківський компонент?
2. Що таке prop drilling і яку проблему вирішує Context API?
3. Що таке reducer? Чим reducer відрізняється від звичайної функції оновлення state?
4. Якою повинна бути чиста reducer-функція?
5. Що таке action і dispatch?
6. Коли `useReducer` доцільніше використовувати за `useState`?
7. Що таке Context API і з яких основних частин складається його використання?
8. Чи потрібно через Context передавати весь state застосунку?
9. Що таке side effect у React?
10. Для чого використовується `useEffect`?
11. Коли запускається Effect і коли виконується cleanup?
12. Чому запис у `localStorage` є побічним ефектом?
13. Чому timer для Undoable Delete не можна створювати всередині reducer?
14. Для чого потрібен `clearTimeout` у cleanup?
15. Чому не варто дублювати derived state?
16. Що таке Custom Hook?
17. Які правила потрібно дотримуватися при створенні власних hooks?